课程笔记：智能体的记忆与技能

Bowen-mantou（整理）

2026-10-23

视频作者/频道：B 站「常青藤名校课」· CMU 11-768 AI Agents（英文原声字幕版）
发布日期：2026-09-09
视频时长：75:22
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=18
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 引言：从「记住本次任务」到「跨任务积累」 2

1.1 经验的三层抽象：episode、fact、skill 2

1.2 人写技能与学习技能的对比 3

1.3 本章小结 3

2 人写技能：Agent Skills 标准与渐进披露 3

2.1 一个技能包长什么样 3

2.2 渐进披露（progressive disclosure） 3

2.3 技能是否真的有用：Skills Bench 4

2.4 如何评估与维护技能 4

2.5 本章小结 4

3 外部记忆系统：MemGPT 与 Mem0 4

3.1 MemGPT：把上下文当「内存」来管理 4

3.2 Mem0：记忆需要被「更新」而不只是「追加」 5

3.3 本章小结 5

4 反馈与轨迹：让 agent 从经验中学习 5

4.1 Reflexion：用自然语言反思做「第二次尝试」 5

4.2 轨迹记忆：「给 agent 的 RAG」 5

4.3 本章小结 6

5 技能诱导：从经验中抽象出可复用子任务 6

5.1 什么是可诱导的技能 6

5.2 历史脉络：Voyager 与 DreamCoder 6

5.3 在线学习设定 6

5.4 Agent Workflow Memory（AWM） 6

5.5 本章小结 7

6 文本技能 vs 代码技能：表示决定能力边界 7

6.1 代码技能的诱导与测试 7

6.2 SkillWeaver：用执行反馈修订技能 7

6.3 效率与成功率：代码技能更强 7

6.4 泛化失败与 PolySkill 7

6.5 三种表示的总账 8

6.6 本章小结 8

7 从失败学习与记忆管理 8

7.1 失败经验也有价值 8

7.2 裁判质量、过度检索与记忆合并 8

7.3 本章小结 8

8 前沿：用强化学习训练「技能诱导器」 **9**
8.1 本章小结 9

9 总结与延伸 **9**

1 引言：从「记住本次任务」到「跨任务积累」

上一讲讲的是**任务内**的上下文管理：在有限的上下文窗口里决定保留什么、压缩什么。这一讲把时间尺度拉长：agent 如何把这次任务的经验带到下一次任务，形成跨任务的**外部记忆**与**技能**。

课堂用购物网站的例子说明动机：任务一是「把 Sony 耳机加进心愿单」，任务二是「查无线键盘的价格区间」。两个任务里有一大段动作是共享的——打开商店、找到搜索框、输入关键词、点击搜索；只有后半段不同。如果 agent 每次都要重新摸索「这个网站的搜索怎么做」，就浪费了经验。于是核心问题变成：**如何表示这种可复用的共享结构**——保留原始动作序列？抽象成可直接调用的代码函数？还是写成一段自然语言说明？

主讲人把「更新 agent」的方式分成三类，并比较了各自的优缺点：

方式	优点	缺点
上下文窗口（上一讲）	保真度最高、实现最简单	成本高、受窗口限制、每步都要重算
外部工件（本讲）	可跨模型移植、人类可检查可编辑	需要被检索/选择/维护，可能过时
模型权重（SFT/RL）	推理快、泛化到行为改变	更新慢、不可解释、只对特定模型有效

1.1 经验的三层抽象：episode、fact、skill

外部存储里可以放三种不同粒度的东西：

1. **episode（完整回合）**：观测、动作、工具调用的完整序列。保真度最高、保留环境与指令细节，但 token 成本也最高；而且大多数细节并不需要——就像监督学习不会记住每个样本的全部细节，而是要抽象出可泛化的东西。

2. **fact（事实）**：关于环境或用户的知识，例如「用户住在某地」「用户的年龄」。早期记忆系统大多聚焦于此：用户和助手交互时透露个人信息，系统存下来，未来用这些事实改善交互。今天 Anthropic、ChatGPT 的「记忆」功能本质就是这一层：模型从历史里生成一段事实文本，存进外部存储。

3. **skill（技能）**：与具体任务细节解耦的「如何做某类子任务」的知识，例如「如何在这个网站搜索商品」。它可以是文本、动作序列，甚至代码；前提是该子任务会被重复执行。

记忆与技能的交集是这一讲的主线：记忆是从 agent 自身交互中保存下来的东西（「上次买过什么」）；技能是可复用的行动知识（「发布到生产前要过哪些检查」）。技能可以由人写，也可以从经验中**诱导（induce）**出来——后者正是本讲后半部分的重点。

1.2 人写技能与学习技能的对比

	技能作为「指令」（人写）	技能作为「学习」（agent 诱导）
作者	个人或组织	agent 自己
来源	既有经验、政策、文档	自身回合与结果
优势	来源清晰、可控、可审计	能发现人说不清的做法、可自动化
负担	写和维护耗时	需要判断、验证与筛选，可能噪声大

两种路线是互补的：可以让人提出候选最佳实践、由 agent 基于经验改进；也可以反过来，让人写的技能被 agent 根据实操反馈精修——形成 human-in-the-loop 的迭代闭环。

1.3 本章小结

跨任务积累有三种载体（上下文、外部工件、权重），外部工件的优势是可移植、可检查、可编辑。经验可以存成回合、事实、技能三种粒度；技能既可由人编写（编码最佳实践），也可由 agent 从经验中诱导，两条路线互补。

2 人写技能：Agent Skills 标准与渐进披露

2.1 一个技能包长什么样

现在工业界的技能标准（agent skills）非常朴素：**一个技能 = 一个文件夹**，其中必须有 `SKILL.md`：

- `SKILL.md` 以 YAML 元数据开头，至少包含 `name` 与 `description`；`description` 决定了模型「什么时候该用这个技能」；
- 元数据之后是详细的正文，写清楚技能该如何执行（要跑哪些 linter、测试怎么做、代码风格等）；
- 可选子目录：`scripts/`（可执行脚本）、`references/`（按需查阅的补充文档）、`assets/`（模板等资源）。

2.2 渐进披露（progressive disclosure）

一个技能包可能很大，不可能把所有技能全文塞进上下文。因此采用三级渐进披露：

1. **第一级：元数据**（name + description）——只要技能可用，就常驻系统提示；因此描述必须短，否则技能一多就把上下文占满；
2. **第二级：`SKILL.md` 正文**——模型判断该技能相关时，通过工具调用（如 `skill_view`）读取；
3. **第三级：资源文件**——模型在执行过程中按需用工具读取脚本、参考文档或资源。

课堂提到一个现实数字：某个 state-of-the-art agent（Hermes agent）的系统提示里，技能描述总长度大约 3,000 token。技能不是越多越好：技能描述变长、技能数量变多，都会占用上下文并可能分散模型注意力；有论文显示加入无关技能会让性能下降。

2.3 技能是否真的有用：Skills Bench

技能研究还很新，但已有较系统的评测：**Skills Bench** 从网上收集了大量技能（课堂口径约两百万条），再让人独立写约 400 个「看起来能用上技能」的任务，并由标注者从技能池中挑选适用技能，提供给 4 个 agent 框架与约 18 个模型，比较「给技能 vs 不给技能」的通过率。主要发现：

- 平均通过率有明显提升（课堂图表口径：约 33.9% 提升到 50.5%）；
- 需要技能较少的任务收益更大；需要很多技能的任务收益反而变小——一部分是难度，一部分可能是模型被大量技能干扰；
- 对部分模型/任务，技能反而伤害性能：模型误用了技能，用与作者意图不同的方式完成任务。

2.4 如何评估与维护技能

课堂给了 OpenHands 的一个代码评审技能案例，形成闭环：agent 用技能产出评审意见 → 人逐条处理（采纳或不采纳）→ 用模型判断每条意见是否被采纳、统计采纳率 → 在成百上千个 PR 上汇总 → 用推理模型总结「哪些类型的意见有用、哪些没用」→ 让模型提出对 `SKILL.md` 的修改。

一个关键经验：**不要凭空写技能，要基于历史交互写**。Skills Bench 显示，只给模型一个任务描述让它自己造技能，效果很差、甚至让性能变差；而基于过去真实交互（以及实操中暴露出的偏好）提炼技能，效果明显更好。

2.5 本章小结

Agent Skills 用「文件夹 + SKILL.md + 元数据」把技能标准化，用三级渐进披露控制上下文占用。技能能提升通过率，但「技能多了会互相干扰、误用技能会变差」是真实风险；技能要基于真实交互提炼，并用「人工采纳率 + 模型汇总」的闭环持续维护。

3 外部记忆系统：MemGPT 与 Mem0

3.1 MemGPT：把上下文当「内存」来管理

MemGPT（2023）是较早系统化提出「上下文窗口之外管理记忆」的工作，思路直接借鉴操作系统：上下文窗口相当于内存（快但小），外部存储相当于磁盘（大但慢），中间用工具调用做换入换出。它把状态分成：

- **系统指令**：固定不变；
- **工作上下文**：从外部记忆检索出来的、当前要用的内容；
- **外部记忆/消息存储**：完整历史与事实存在外部，读写都由模型发起的工具调用控制。

课堂用「用户提到 Six Flags（游乐园）」举例：模型发一个工具调用去外部存储里检索历史上关于 Six Flags 的信息，把结果放进工作上下文，再基于它回答。在上下文窗口只有 8K 的年代，这种做法对普通助手都很有必要；今天窗口变大了，但 agent 要存的东西也变多了，思路依然成立。

3.2 Mem0：记忆需要被「更新」而不只是「追加」

只追加的记忆会过时：用户先说「我在 X 公司工作」，后来说「我换工作了」，系统不能两条都信。Mem0 给出了记忆维护的流程：

1. 每条消息进来后，用 LLM 抽取候选记忆（事实）；
2. 用 embedding 检索数据库中最相关的 top-K 记忆；
3. 用 LLM 对比「新抽取的事实」与「已存的相关记忆」，决定四种操作之一：**ADD**（新事实）、**UPDATE**（旧事实被更新）、**DELETE**（旧事实被否定，如「我被裁了」）、**NOOP**（无关，忽略）。

这个「先检索、再决定增删改」的结构，与上一讲线性注意力/DeltaNet 的「状态矩阵更新」在思想上同源：普通线性注意力只能追加，DeltaNet 会针对 query 检索并修改已存内容；Mem0 是文本层面的类比。同样的「遗忘/更新」问题在权重层面也存在——这正是 unlearning（机器遗忘）研究的动机。

3.3 本章小结

MemGPT 把上下文当内存、外部存储当磁盘，用工具调用换入换出；Mem0 补上了记忆的「维护」——抽取、检索、增删改。记忆系统的核心难题不是「存得下」，而是「冲突时以谁为准」以及「什么时候该忘」。

4 反馈与轨迹：让 agent 从经验中学习

4.1 Reflexion：用自然语言反思做「第二次尝试」

Reflexion 是一篇简单但影响很大的工作：agent 对一个任务尝试一次后，由评估器（人、规则或模型裁判）判断结果是否正确；如果不对，就生成一段**自然语言反馈**（哪里错了、下次怎么改），再带着这段反馈重试。

课堂例子：问「某系列战役包括哪些」，模型只答了其中一场；反馈指出「你只给了一场战役而不是一系列」，模型据此在第二次尝试中修正。论文表明，这种自然语言反馈作为条件，能显著提高第二次尝试的成功率。其价值还在于可推广：只要反馈足够通用，它不仅能改进同一个任务，也能帮助相似任务。

4.2 轨迹记忆：「给 agent 的 RAG」

更进一步是把过去**完整轨迹**存起来：收集一批训练任务（可以是人类示范，也可以是 agent 自己跑出来的成功/失败轨迹），必要时为每条轨迹生成反馈文本；新任务到来时，用任务描述的 embedding 去检索相似轨迹，把轨迹（或摘要）放进上下文当作 few-shot 示例。这本质上就是**面向 agent 的 RAG**——检索单位从「文档」变成「任务轨迹」。

课堂讨论的细节：轨迹 = 环境/用户消息（观测）+ 动作（工具调用）+（可选的）思维链，基本等同于「上下文窗口里所有消息的序列」。检索通常按任务描述做 embedding，也可以先按领域约束（只检索同一网站/同一类型任务的轨迹）。

4.3 本章小结

Reflexion 证明「自然语言反思 + 重试」是提升 agent 表现的低成本手段；轨迹记忆则把过去的完整经验当作可检索的示例（agent 版 RAG）。两者都在回答同一个问题：如何把「一次性的经验」变成「下次可用的条件」。

5 技能诱导：从经验中抽象出可复用子任务

5.1 什么是可诱导的技能

技能 = 任务中可跨任务复用的片段。购物例子中，「搜索某商品」这个片段会在「加心愿单」「查价格」等不同任务中反复出现；把这段动作序列抽象成带参数模板（例如把搜索词抽象成变量），就得到一个技能。围绕技能有四个关键环节：

1. **表示**：保留原始动作？抽象成文本模板？还是写成代码函数？
2. **诱导**：从哪里、用什么信号把技能抽出来；
3. **入库**：判断该轨迹是否成功、技能是否值得存；
4. **生命周期**：技能太旧/太具体/太多时，如何合并、泛化、删除。

5.2 历史脉络：Voyager 与 DreamCoder

- **Voyager**（Minecraft）：用**代码函数**表示技能（合成物品、打僵尸等），函数之间可以互相调用；agent 学会简单函数后可以组合出越来越复杂的行为。
- **DreamCoder**（程序归纳）：在构造对象的语法上搜索重复模式，把反复出现的子结构抽象成函数，从而「压缩」程序；后续工作又引入 LLM、Python 代码与自然语言描述。这些思想今天被用来做 agent 的技能库。

5.3 在线学习设定

技能诱导通常在**在线学习**设定下评测：任务一个接一个到达，agent 在解决任务的过程中增量地建立技能库；每个任务都测成功率，看成功率和可完成任务的复杂度是否随经验增长。技能库的完整生命周期是：**诱导（learn）→ 判断与入库（store）→ 使用（use）→ 精简/合并（simplify）**。

5.4 Agent Workflow Memory（AWM）

CMU 的工作（Zora Wang 等）在 Web 任务上做文本技能诱导：拿到一条成功的工具调用轨迹后，用 LLM 识别其中「未来可能复用」的片段，输出 **workflow**——一段文本描述 + 动作序列模板（把具体搜索词等抽象成参数）。还要用一个**裁判模型**判断这条轨迹是否真的成功；只有判定成功的轨迹，其 workflow 才入库，避免把错误经验教给未来。

使用时，workflow 直接放进上下文（类似 agent skills 的渐进披露：可以只放描述，也可以放完整动作示例）。实验结果：在交互式地图工具的任务序列上，无记忆 baseline 的成功率停留在 10% 量级，而带 workflow 记忆的 agent 成功率明显上升，并且能做越来越复杂的任务（如「找附近的酒店并给出步行路线」），差距随任务数量扩大。

5.5 本章小结

技能诱导把「任务片段」抽象成可复用表示（文本模板或代码），关键四步是表示、诱导、入库判断与生命周期管理。AWM 用「LLM 抽 workflow + 裁判过滤 + 放回上下文」证明技能记忆能带来跨任务复利；Voyager/DreamCoder 则提供了「用代码/程序归纳表示技能」的思想源头。

6 文本技能 vs 代码技能：表示决定能力边界

6.1 代码技能的诱导与测试

同一段技能可以表示为「文本 + 动作示例」，也可以表示为「代码函数」。代码路线的代表工作（Agent Skill Induction / ASI）做法是：给 LLM 一条成功轨迹，让它直接生成抽象函数（含 docstring，内部就是低层工具调用），并生成一条「用新函数重写后的轨迹」。相比文本技能，代码技能多了一个关键优势：**可以执行、可以测试**。校验从「让裁判看原轨迹」升级为「执行抽象后的代码、再让裁判看最终结果」——信号更强。

代码技能的收益还包括：函数可嵌套、可分层（技能调用技能）、可以单独写测试、可以重构。代价是**僵硬**：模型调用这个函数时只会执行固定的几步，没有偏离空间。

6.2 SkillWeaver：用执行反馈修订技能

SkillWeaver 把代码技能的闭环做成一个循环：给定任务 → 让模型提出候选代码技能 → 执行任务（得到 episode）→ 用奖励模型判断是否成功 → 成功则把技能写入记忆，失败则修订技能。修订信号可以来自 **linter**：论文里的真实例子是「identify pill」技能声明了颜色参数却在函数体里没用上（忽略 `color` 参数），linter 发现未使用参数，模型据此修订。也可以把奖励模型的反馈作为条件来改技能。

6.3 效率与成功率：代码技能更强

代码技能还有一个重要优势：**显著减少与环境的交互次数**。课堂给了一个真实例子：在一张页面上更新两个地址，无记忆的 agent 要一步步导航、填表、提交；而拥有「进入地址设置」「更新地址详情」两个函数技能的 agent，三步就能完成任务。由于 GUI/浏览器 agent 的每一步都要处理图像与思维链、token 成本很高，减少交互步数对效率意义重大。

在重复结构较多的任务集上做对照实验：无记忆 < 文本技能 < 代码技能（成功率）；代码技能还大幅降低了每任务的交互步数。

6.4 泛化失败与 PolySkill

技能最大的风险是**不泛化**。课堂给出真实的失败案例：在一个网站上诱导出的代码技能，拿到 Target 购物网站上就失效——原站点用下拉框选择，Target 用一组单选按钮，元素 ID 也完全不同，技能直接报错。解决办法来自软件工程：**抽象类 + 具体实现**。PolySkill 让模型第一次接触新网站时写一个抽象类（定义 search、add_to_cart、checkout 等接口的桩），再写该网站的站点实现；换站点时只需要新的实现，并按元数据或模型判断激活对应实现。这样既能保存大量站点特定的技能，又只让当前相关的那一小部分生效。

6.5 三种表示的总账

课堂上用一张表总结了「检索到的回合 / 文本技能 / 代码技能」三种表示：

表示	优点	缺点
检索到的回合	保留完整行为、保真度最高	长、难以迁移到新任务
文本技能	灵活可适应新页面、可组合	不提升效率，仍需模型逐条发动作
代码技能	可执行、可测试、可组合、效率高	脆弱：页面一变就失效

6.6 本章小结

表示决定边界：文本技能灵活但不省交互，代码技能高效、可测试、可组合但僵硬脆弱。工程解法是把两者结合，并用抽象类/多实现（PolySkill）处理跨站点泛化；任务重复度越高，代码技能的收益越大。

7 从失败学习与记忆管理

7.1 失败经验也有价值

前面所有讨论都在「学习成功」，但失败经验同样可以转化成技能。课堂例子：在某网站上搜索无线耳机时，站点把多个词按「或」而不是「与」处理，于是返回 5,000 条结果，agent 翻页翻到迷失、任务失败。若把这条失败轨迹用类似 Reflexion 的方式总结成文本策略——「用更具体的搜索词、让每页显示更多条目」——它就能作为技能指导未来任务。

论文的对照发现：**负例对「文本策略」型记忆有显著帮助，但对「完整轨迹」和「workflow」型记忆没有帮助**。原因很自然：轨迹/工作流是「照着做」的示例，负例并不能直接转化为可执行的动作；而文本策略可以描述「如何避免失败」。

7.2 裁判质量、过度检索与记忆合并

技能/记忆入库依赖「这条轨迹是否成功」的判断，而裁判并不完美。ReasoningBank 做了受控实验：用真实标签按一定概率翻转来模拟不同准确率的裁判，结果性能随裁判准确率下降而下降，但存在一个**甜点区**——在相当宽的裁判准确率范围内，用裁判诱导记忆仍然有益。

检索也有代价：ReasoningBank 的消融显示，**检索更多经验反而会让性能下降**。原因可能有两种：任务本身与记忆里的经验相似度不够（存了也用不上）；或者模型被上下文中无关内容干扰。一个直接的工程解法是**主动缩小记忆**：按使用频率淘汰——某函数被使用的次数少于 $\lambda \log(t)$ 量级（t 为见过的样本数）就丢掉；这与缓存淘汰的思路一致，既省上下文又可能提升成功率。

7.3 本章小结

失败经验对文本策略型记忆有效，对轨迹/workflow 型记忆无效；裁判质量决定记忆质量，但有较宽的容错区间；检索越多不一定越好，按使用频率淘汰记忆是有效的工程手段。

8 前沿：用强化学习训练「技能诱导器」

前面所有方法都有一点「手工感」：诱导技能、决定是否入库、选择保留哪些，都是通过提示词让 LLM 做的。一个自然的问题是：能不能直接**训练**模型做技能诱导？

近期的方向是在「相似任务序列」上做强化学习：任务依次到来，agent 诱导技能、建立记忆、在后续任务中使用记忆；奖励来自两方面——后续任务是否成功，以及**是否真的复用了先前诱导的技能**。这类过程包含多个模型、外部存储与检索，完全不可微，但 RL 不需要可微：只要拿得到最终奖励，就可以把整条流水线当作策略的一部分来优化。

课堂给出的一篇论文结果：只奖励任务结果时 55.4%，两个任务都成功时 56.6%，加入「成功复用技能」的奖励项后提升到 60.7%（图口径）。也可以预见更多奖励项的设计，例如惩罚技能库过大、鼓励技能精简，以防过拟合。

8.1 本章小结

把技能诱导交给 RL 是刚出现的方向：过程不可微没关系，只要有最终奖励。初步结果显示「任务成功 + 技能复用」的组合奖励有效；奖励设计与技能库正则化仍是开放问题。

9 总结与延伸

本讲把「记忆」与「技能」放在同一条演进线上：上下文窗口内的管理（上一讲）→ 跨任务的外部记忆（事实/回合）→ 技能（可复用的子任务表示）→ 技能的自动诱导与生命周期管理。核心要点可以压缩为：

1. 更新 agent 有三条路：上下文、外部工件、权重；外部工件可移植、可检查、可编辑，是技能与记忆的天然载体。
2. 人写技能有标准（Agent Skills：`SKILL.md` + 元数据 + 资源目录）与三级渐进披露；技能有用但「过多/误用」会伤害性能。
3. 记忆系统从 MemGPT（换入换出）走到 Mem0（增删改与冲突处理）；核心问题是冲突时以谁为准、何时遗忘。
4. 技能可以用文本或代码表示：文本灵活、代码高效可测试但脆弱；抽象类 + 多实现（PolySkill）是跨站点泛化的工程解。
5. 技能诱导要过滤失败轨迹、控制检索数量、按使用频率淘汰；裁判质量重要但容错区间较宽。
6. 最新方向是用 RL 端到端训练技能诱导器，奖励包含「任务成功」与「技能复用」。

留下来讨论的三个问题（课堂原题）

1. 你会把经验存成回合、事实、经验教训、文本技能还是代码？为什么？
2. 何时应该删除或修改一个技能？
3. 在「人写技能」与「agent 自动诱导」之间，人与机器的分工应该怎么划分？

延伸思考：(1) 记忆与技能的边界会随着「表示能力」变化——当代码技能足够强时，一个技能库其实就是一个可调用的工具生态；(2) 从系统角度看，技能库 缓存 + 检索 + 版本管理，缓存命

中率、失效策略、版本兼容性这些老问题都会重现；(3) 从研究角度看，如何表示「人机共同维护的技能」、如何避免记忆污染与技能过拟合，仍是开放问题。下一讲进入「规划、任务分解与多 agent 协作」，讨论 agent 如何把复杂目标拆解成可执行的结构。